Interactive Chatbots: a new approach from turn-based LLMS

Aryan Amberkar *aryanka2@illinois.edu*
Vihaansh Majithia *vjm5@illinois.edu*
Hersh Doshi *hershd2@illinois.edu*
Hanshul Bahl *hbahl2@illinois.edu*

Abstract

This paper goes over the final project for Group 17 which involves creating a chatbot which can interrupt users. This project draws inspiration from the paper published by the University of Texas at Austin (UT Austin), Microsoft Research, and Amazon Web Services (AWS). Essentially this paper aims to provide a solution by having three components, a verification model, a speculative model, and a response model. The verification model detects when to interrupt, the speculative model completes the partial user input from the verification model, and the response model provides the response to the output of the speculative model. We then evaluate these models and discovered the verification model was quite accurate, but the speculative model could use some more fine-tuning. However, the results do prove that an interruptible chatbot can be created and can be accurate in certain use cases.

1 Introduction

Conversational AI systems are becoming increasingly prevalent in both personal and enterprise settings, powering virtual assistants, customer support platforms, and productivity tools. While modern large language models have achieved very high fluency compared to human interaction, they are still predominantly designed to operate in a turn-based interaction model, where the user completes their input before the system generates a response. This fixed structure does not allow the LLM to mimic natural human dialogue, where participants frequently interrupt or overlap while talking to clarify, correct, or advance the conversation.

The lack of interruption capability introduces limitations that might hamper realism and may annoy the user, especially when trying to correct the chatbot. For instance, in collaborative problem-solving, a user may want the chatbot to stop early if it is providing irrelevant details. In real-time meetings, an assistant may need to proactively supply facts or corrections before the discussion moves on. However, enabling such real-time conversational cut-ins requires tackling core challenges at the intersection of systems and human-computer interaction.

Current solutions only partially address the problem of real interruptions in chatbots. Traditional turn taking models such as TurnGPT or Voice Activity Projection predict when a speaker's turn ends, improving timing but not deciding whether an interruption adds value. Incremental dialogue frameworks process user input as it arrives, this allows for overlap or early responses, however, they lack system design and lack learning based policies that weigh usefulness or politeness to make chatbots more human. Human computer interaction research on proactive or spoken interruptions offers design heuristics like signaling urgency or using "excuse me" rituals, but this doesn't translate into adaptive decision models where it actually decides where its important to cut in. Furthermore, solutions on text based overlap improves interactivity in typing scenarios but doesn't handle the contextual appropriateness of when to intervene. Meanwhile, streaming LLM architectures like StreamingLLM make continuous, low latency processing possible but don't tackle the behavioral or policy aspects of interruption. Hence current solutions optimize when to speak or how to stream, but not why or whether a chatbot should cut in.

2 Motivation

The motivation behind a chatbot with the power to potentially interrupt conversation is that it allows for a chatbot to be more interactive. Often in human conversation, we see that it is rarely dominated by one-side. Another member in the conversation may interrupt at any time, and this adds more of a "human" feel to the conversation. Now this raises the question, "When does a chatbot feel it is necessary to interrupt?". This is an interesting question that is at the core of our project.

Naturally, in normal conversations, we expect one to interrupt to ask clarifying questions, or provide additional context. The chatbot would have to be able to recognize certain areas of ambiguity, and directly cut-in, mimicking a human-style conversation. It would have to analyze certain situations where the user's input may diverge from the expected conversation flow, and at this point it may interrupt. This forced attention causes the chatbot to

be more interactive, as the chatbot is now focused on delivering the right response, while also attending to this matter. This is an important feature because this directly leads to a more attentive chatbot.

From a user experience view, a chatbot that can directly interrupt may feel more responsive. Offering help at certain moments or providing guidance, can help users feel more engaged and offer a better experience all around for the user.

Furthermore, consider the situation where a user is using a chatbot to learn a particular topic, if the user continues to make incorrect assumptions, the chatbot can directly correct the user, instead of waiting for them to continue on with their message.

The current model involves a turn-taking approach, where the user initiates the conversation, the chatbot responds, and this cycle is then repeated for the conversation length. Perhaps, while a user is typing a message, the chatbot prompts the user with a message, this prompting would require the user to re-evaluate their message. Perhaps if a user is typing for a long time, the chatbot could sense this and interrupt, offering to clarify certain concepts. Since the current turn-taking makes dialogues feel mechanical and delays responsiveness. However, human conversation is highly incremental in a sense where people overlap in speech about 40 percent of the time and interrupt roughly once every 60 seconds in collaborative settings. Furthermore, studies in human-robot and human-agent interaction confirm that delays longer than 500 ms between user input and system response noticeably reduce perceived naturalness and engagement. These timing gaps make current chatbots appear inattentive or slow to react.

Current research on turn-taking and overlap further supports the need for proactive behavior. Allowing text-based overlap increased user satisfaction scores by 23 percent and reduced perceived latency by 35 percent, even when total task time remained constant. Also, well-timed speech interruptions improved task completion speed by up to 27 percent compared to strictly turn-based systems. These findings suggest that timely cut ins can make dialogue feel smoother, more efficient, and more “alive.”

Also, from a systems standpoint, the growing availability of low latency streaming LLMs makes real time interventions technically feasible, for example, StreamingLLM and related architectures process tokens continuously with latencies under 100 ms per token. Yet there isn’t a system on when and where to cut in, we aim to develop an cut in mechanism that decides when interruption is both contextually appropriate and informative, directly addressing this issue.

This project aims to make chatbots dialogue as natural and responsive as human conversation, and to also quantitatively validate that intelligent interruptions can enhance success in models and result in better outputs, as well as have better user engagement.

3 Background & Related Work

Based on the suggestions of the professor, there were two papers highly relevant to the project. The first is the paper "Speculative Ad-hoc Querying" published in a collaborative effort between the University of Texas at Austin, Amazon Web Services, and Microsoft Research. The second paper is "Single-agent or Multi-agent Systems? Why Not Both?" published by the Siebel School of Computing and Data Science at the University of Illinois Urbana-Champaign.

3.1 "Speculative Ad-hoc Querying" Paper

The first paper, "Speculative Ad-hoc Querying" raises some important and relevant points to our research project. The premise of this paper stems from the idea that LLMs can be used to predict future queries of the user, with the hope that eventually one of these precomputed queries performed by the LLM will yield a match in any future queries. This is analogous to the problem statement of our research project. We wish to create a model which can accurately predict when to interrupt the user based on prior words, making the chatbot more interactive.

The paper defined three levels of speculative execution. Level 0 indicates that the predicted query by the LLM is an exact match to that of the user. In this case, the model can directly output the result from its precomputed execution list. Level 1 indicates the model can reasonable guess what subset of the data the user may wish to further analyze, and create sub-tables based on certain characteristics. Level 2 indicates that the model has a reasonable understanding of where to proceed given prior queries. It can create relevant pre-computed tables that match the general overview of what the user may want to analyze. This line of reasoning can be implemented in our chatbot project. Realistically, we would strive for the chatbot to have at least "Level 1" understanding of the user prompt in order to interrupt, however this is still up for debate.

The paper then further mentions the use of a speculator. The purpose of the speculator is to "speculate"; given the current, incomplete query of the user, speculate a potential query that best matches their desired goals, based on previous inputs. The relevant components to our project include the "Auto-completion" and the "Over-projection" features of the paper. Auto-completion essentially refers to the task of predicting what the user may type given their incomplete query. Over-projection is the process by which the model overestimates the user’s query by adding more columns and tables to the query. This allows to model, with a higher probability, to ensure that

its precomputed query can possibly contain some level of matching with that of the users'. The use of over-projection in our chatbot could be an interesting feature. This would essentially mean that whenever the model interrupts, it could provide extra information in its response to elicit a "positive" (good interruption) response. For example, the user could be typing, "Alaska is a state...", and the model could potentially respond in the following way, "Alaska is a state known for being the largest in the United States, and is also famous for its vast nature parks and wildlife". By providing extra information in the response, we can maximize the chances of a "good interruption".

The evaluation process for this paper essentially involved using pre-existing queries to see if the model was able to predict a viable alternative to the user query. Then using mathematical formulations, the latency and cost cut was derived.

Overall, this paper did give a good start to our chatbot project by going over the levels of speculation, and the speculator and the over-estimator.

3.2 "Single-agent or Multi-agent Systems" Paper

The paper, "Single-agent or Multi-agent Systems? Why Not Both?" covers an essential topic, which is as modern LLMs get stronger, when is it actually worth using a multi-agent system instead of a single agent? After the authors tested 15 tasks which include coding, math, planning, RAG QA, finance, and science, the authors found that the MAS's historical accuracy edge shrinks with newer base models, while costs stay steep, where they are often 4-220x more prefill tokens and 2-12 times more decode tokens than SAS. On easy and middle-tier problems, SAS typically matches MAS and MAS helps mainly on the toughest, highly compositional cases, and even there, the margin narrows as base models improve.

To explain when MAS underperforms, the workflow is viewed as a dependency graph with three failure modes. There are node-level defects where one "critical" agent becomes a bottleneck that caps overall quality, making further decomposition pointless. There also exist edge-level defects where downstream agents get flooded by long or conflicting messages, such as an "overthinking via inputs" issue, which is distinct from simple chain of thought drift. Finally, there are Path level defects where information erodes along long chains, so a correct intermediate step never reaches the final answer. These dynamics produce many ties, some SAS wins due to cleaner paths, and MAS wins mostly when decomposition is truly necessary.

The paper then proposes two practical fixes for cost-effective deployments. First, a confidence-guided critical-path tracer that rewards agents based on final correctness and self-reported confidence, showing the true bottleneck to upgrade rather than overhauling the whole MAS. Second, a hybrid SAS MAS strategy where there is difficulty-based routing that sends simple queries to SAS and hard ones to MAS, and a cascade that tries SAS first and escalates to MAS only when a verifier flags low quality. Across tasks, these tactics raised accuracy by 1-12 % while cutting costs by 20-88 %, recovering most of MAS's upside at near SAS efficiency and aligning with interruptible/speculative designs that escalate only when evidence warrants it.

4 Design

Our solution combines three different models, the verification model, the speculative model, and the response model. As a brief overview, the verification model predicts when to interrupt a user. It will only interrupt once it is confident it has enough prior knowledge and can accurately gauge the remaining prompt from the user. The speculative model is responsible for essentially completing the prompt of the user. So if a user types in "The lion is the king", we would expect the speculative model to produce something like "The lion is the king of the jungle". The response model would be responsible for providing the response to the prompt completed by the speculative model.

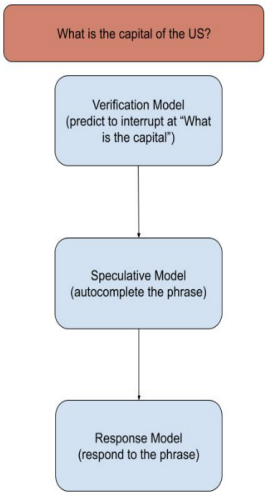

Figure 1: Overview of Solution

In figure 1, we see how the user's original prompt is "What is the capital of the US?". The verification model will predict to interrupt at the phrase, "What is the capital". This input is then fed into the speculative model, where it will auto-complete the phrase. The response model then responds to the query.

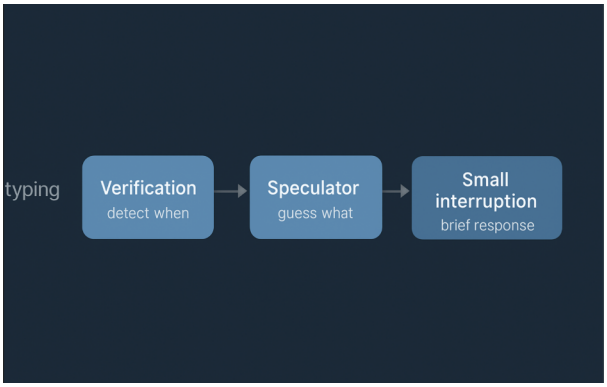

Figure 2: Overview of Solution

Figure 2 provides another detailed image describing our implemented solution.

5 Implementation

This section goes into detail on how the three individual components were created, and how they were combined into one final chatbot.

5.1 Verification Model

As discussed earlier, the verification model is responsible for predicting when to interrupt. The goal is to interrupt when we have enough information from the input/query of the user, and if we are confident that their input is a valid/plausible beginning to a sentence.

In detail, the verification model was trained on two sets of data. The first set of training data involved the WikiText dataset. The reason this dataset was used was because it provides us with a lot of general English sentences which can allow the model to learn how to recognize and identify common prefixes to a sentence. The WikiText dataset contains various articles from Wikipedia and allows the model to capture and learn a variety of sentences in the English language. The second dataset was the Stanford Question Answering Dataset (SQuAD). For this dataset, we only used the question portion, as we wanted to model to recognize valid sentences. The SQuAD dataset contains questions for certain Wikipedia articles, and their corresponding answers. By forcing the model to pay attention to questions, it can recognize valid prefixes/beginnings to questions. This two-layered approach of training on both questions and sentences was chosen because we wanted the model to be able to interrupt for both sentences and questions.

Now we go into details on how the verification model was implemented. Essentially the model is trying to answer the question: "Given this partial input from a user, is it valid/correct to interrupt the user and respond now?". So we want to essentially make a choice of interrupting or not interrupting for every prefix the user enters. As the user types in a word in each query, the verification model will choose of interrupting versus not. The model must be able to understand both syntax and semantics as we need to identify if this partial input is valid and constructive. Hence, we decided to use DistilBERT, which is a smaller version of BERT. DistilBERT allows us to use a transformer encoder to capture this language structure. Now as explained in the paragraph above, we trained the model on two datasets. During the training process, for each sentence/question, we generated a random cutoff point. The cutoff point would be from the range of 3 to the length of the training input - 1. We then took the substring of the training input from 0 to this cutoff point and assigned it an "interruptible" tag. This allowed the model to recognize valid interruptible prefixes. We also trained the model on prefixes that it should not interrupt on. We never want to interrupt on prefixes with a length of 1-2 sentences (not enough info) and for random words with no coherent meaning. To do so, we fed the model three different versions of sentences/questions during which we should never interrupt:

1. Version 1: Random Word Order

   We took the training input and randomly shuffled the words. This allowed the model to recognize that even words with coherent meaning can create an "incorrect, uninterruptible" prefix, if the order makes no sense.

2. Version 2: Middle of Sentence Words

   We took the cutoff point and simply fed the the model the words from the cutoff point to the end. This gives us a sentence but doesn't begin with anything concrete. For example, if we have the phrase, "What is the purpose of backpropagation in neural networks, and how can this be optimized through features like Adam or SGD?", if we merely take the middle of the sentence such as "how can this be optimized", the snippet is a grammatically valid one, but does have any concrete subject. Therefore, the model would not be able to interrupt.

3. Version 3: Short Prefixes

   We never want to interrupt the user when there are only 1-2 words present. It does not make too much sense to interrupt the user when they have only typed in something like "The" or "What" or "How to". We want to interrupt only when we have some level of context which cannot be accurately determined with only 1-2 words.

Now during inference, we process each word the user types in. Each prefix is evaluated by our verification model. The verification model, for each word typed in by the user tells us if it is interruptible or not. So it proceeds incrementally (word by word). For example for the question, "What is the capital of the US?", it will output probabilities of interrupting for each substring:

1. "What"
2. "What is"
3. "What is the"
4. "What is the capital"

Since prefix predictions can be noisy, especially for a shorter prefix, we utilize an exponential moving average to smooth the output probabilities. This smoothing function allows us to stabilize the probabilities better. Once this smoothed probability exceeds a threshold (defined hyperparameter, can be adjusted or fine-tuned) we trigger an interruption and stop.

```
Sentence: What is the capital of the US?
Interrupt at word 4 ('What is the capital')

Sentence: Why is gradient descent used in neural networks?
Interrupt at word 4 ('Why is gradient descent')

Sentence: The lion is the king of the jungle
Interrupt at word 6 ('The lion is the king of')

Sentence: Hello yes the where that what?
No interrupt for this sentence.
```

Figure 3: Example of Verification Model Output

In figure 3, we see an example of the verification model output. For our first sentence, we interrupt after the phrase, "What is the capital", indicating that our model has enough information to interrupt and answer. We can see a similar situation in the second input, as we interrupt after "Why is gradient descent?". The third input is unique in that it is a sentence (like a declaration), and the model recognizes this and interrupts after "The lion is the king of". For the last sentence, we can see that a combination of random words does not yield a valid interrupt, showing how the verification model has learning syntax and language meaning.

5.2 Speculative Model

The speculative model is responsible for completing the user query based off where the verification model predicted to interrupt. We want to create a plausible query given what the user has already entered.

We utilized a T5 (Text-To-Text Transfer Transformer) because T5 specializes in having both the input and output as text strings. T5 models are often used for NLP tasks like summarization or translation, hence we thought it would be an ideal choice for our speculative model. Our speculative model deals with a similar idea as we are given an input as a string, and wish to output the completed query as a string. Furthermore, T5 is an encoder-decoder transformer, which is beneficial for our task. The encoder builds a representation of the prefix entered by the user, and the decoder generates the completed query. By doing this encoder-decoder separation, the model can fully understand the incomplete user query, and then generate a plausible query.

For training the speculative model, we again used the WikiText and SQuAD datasets, for the same reasons as mentioned in the verification model section. We also use a cutoff point, similar to the verification model training process. Training involves using this cutoff point to indicate when the verification model may decide to interrupt. The words from the beginning of the training input to the cutoff point serve as our input prefix. The output of the model is a JSON string with two fields, the produced speculative query, and another query which is the over-speculated query, containing more information. The over-speculated query is inspired from the UT Austin, Microsoft, and AWS paper mentioned in the Background section, and this allows us to include more information in our speculated prompt. Essentially, the speculative model is given text, and trying to produce text given this incomplete query.

During inference, the speculative model is the second of the three models (verification, speculative, and response). At the point of interruption (generated by verification model), the speculative model will take the partial user input, and try to generate an output based on that. The output generated by the speculative model is then sent to the response model, where we can get our final response. We do only use the completed user query instead of the over-projected one, just because the over-projected user query sometimes creates extraneous words with no relation to the original prompt. This is a good feature we could potentially add in the future though.

In figure 4, we can see the outputs for various incomplete user questions, and their corresponding completed queries. We see how the speculative model tries to generate a completed query which could be a plausible question that a user may ask. In figure 5, we see the speculated response to a general sentence, showing how the model can be used for both questions and sentences/declarations.

```
Prefix > What is the largest state

completed:
What is the largest state of the world?

Prefix > What is the smallest animal

completed:
What is the smallest animal in the world?

Prefix > What is the most important

completed:
What is the most important aspect of the world?

Prefix > What is the capital

completed:
What is the capital of the United States?

Prefix > What is the purpose of

completed:
What is the purpose of the project?

Prefix > Where is the best place

completed:
Where is the best place to live?, including a brief explanation.
```

Figure 4: Example of Speculative Model Output

```
Prefix > Water shortages and global warming

completed:
Water shortages and global warming, a major concern for the public.
```

Figure 5: Example of Speculative Model Output for Sentence

5.3 Response Model

The response model is the final component of our implementation, and its purpose is to provide a response to the generated output from the speculated model. Because the response model is a general purpose LLM, we used a pretrained LLM (DeepSeek-V3.2) which is accessed by Hugging Face's OpenAI API. The reason we decided to use a pretrained LLM instead of developing our own was because our project is focused on the interrupting process and finding viable ways to interrupt and generate an output from a partial input. It isn't necessarily focused on creating a chatbot which has knowledge on a wide variety of topics from scratch, which is why we chose this approach. We merely wanted to ensure that we could get an accurate response given the outputs of our verification and speculative models.

Figure 6 provides a diagram illustrating how the response model functions. And figure 7 provides an example output for the response model.

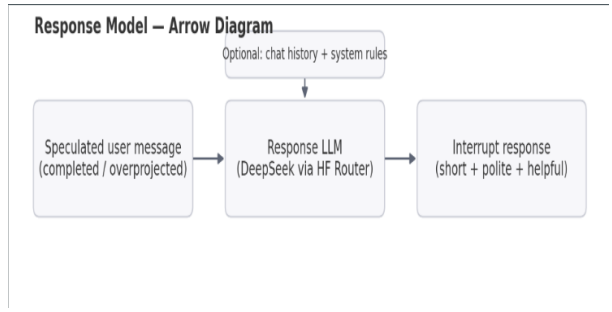

Figure 6: Response Model Diagram

```
> What is the capital of the US?
The capital of the United States is Washington, D.C.
> What is the largest state in the world?
The largest state in the world by total area is Russia.
```

Figure 7: Response Model Example Output

5.4 Final Solution - Combining All Models

The final solution is merely the combination of all three models (verification, speculative, and response) into a functional chatbot.

```
Interruptible Chatbot (3-model pipeline)
Type your message in chunks (like you're typing).
Commands: /send (force response), /reset, /quit

You > What is the capital of the US?
i= 3 score=0.978 smooth=0.587 | What is the
i= 4 score=0.979 smooth=0.822 | What is the capital

============================================================
[INTERRUPT] at word 4 (EMA=0.822)
Prefix: What is the capital
What is the capital of the United States?

Assistant:
The capital of the United States is Washington, D.C.

It is important to specify "D.C." (which stands for the District
of Columbia) to distinguish it from the state of Washington, whic
h is located on the country's Pacific coast.
============================================================
```

Figure 8: Final Chatbot Example Output

In figure 8, we see the example output our chatbot provides. The chatbot first takes in the full user query ("What is the capital of the US?"). Given this query, first the verification model runs. We see that it predicts to interrupt at the phrase "What is the capital". This incomplete partial phrase is given to speculative model which then essentially performs an "auto-complete" on the input. We see that this output is then fed into the

response model, where we get our final answer of Washington D.C. For certain questions, the speculative model may also be able to elaborate, allowing the response model to provide a more robust answer. Consider the example in figure 9. The initial prompt is "What do bears eat in North America?", and given the interruption, the speculative model generates the phrase "What to bears eat?, including a brief explanation.". By adding the phrase "including a brief explanation", this allows for the response model to generate a more robust, detailed example.

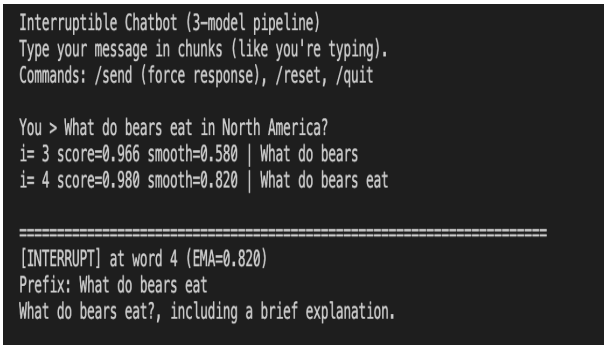

Figure 9: Final Chatbot Example Output #2

6 Evaluation

The evaluation for our project is split into two parts. The first part involves evaluating the verification model and the second part involves evaluating the speculative model. The reason for this separation during the evaluation process is because we wanted to individually evaluate each of the components to identify weaknesses, and potential areas for improvement. This would be a bit harder if we were to evaluate the entire chatbot as a whole. Furthermore, the reason for not evaluating the response model was because the response model simply uses a pre-trained DeepSeek model, hence we assumed that this has already been rigorously tested and is fairly accurate.

6.1 Verification Model Evaluation

Evaluating the verification model involved sampling sentences from the WikiText validation dataset. We used sentences with a length of at least 10 words because we wanted to evaluate on sentences with some level of complexity, and the easiest way to ensure this is through sentence length. Now for each sentence, we create three unique prefixes/inputs:

1. "Valid, interruptible prefix"

   Similar to the training of the verification and speculative models, we have a cutoff point which is randomly chosen. We then take a substring of the sentence from 0 to the cutoff, and label this as a interruptible prefix/input.

2. "Not interruptible prefix"

   This is the same prefix as 1), but we randomly shuffle the words, making it a sentence with valid words, but with no coherent meaning.

3. "Not interruptible prefix"

   We take a random offset from the sentence, and then take a substring of that offset to the end. This will also produce a sentence with no coherent meaning, that should not be interrupted.

We then pass these prefixes into the verification model. If the probability of interrupting is greater than 0.5, then we assign it a valid, interruptible prefix. Otherwise, we assign it as in uninterruptible prefix. Using this approach, the verification model had an accuracy of roughly 0.8667 and a precision value of roughly 0.775.

6.2 Speculative Model Evaluation

Evaluating the speculative model involves a different approach. Essentially for the speculative model, evaluation is performed by measuring the similarity scores between the generated sentence of the model, and the actual sentence. For evaluation, we generated test data from our two datasets, WikiText and SQuAD. For the same reason as the verification model evaluation, only sentences exceeding a certain length are chosen, just for complexity purposes. We then generate a random cutoff point, and then get the substring from 0 to this cutoff point. This prefix is then fed into the speculative model, and the speculative model then generates an output. We then take the similarity score between the output and the original input. A similarity score is essentially a measure between two strings to see how similar they are. We utilize Python's *difflab.SequenceMatcher* to calculate the similarity score. Essentially *difflab.SequenceMatcher* works by finding the longest matching substring that is contiguous between the two strings, as such this is a valid way to evaluate our speculative model.

The similarity score during the evaluation was quite low. Our intuition for this was because the speculative model size was rather small, hence we received a lower accuracy. Also we could make our model more complex to better handle and represent the complexities of language. It is also important to note that language is extremely large and complex, and it is rather difficult to accurately generate an output that is very similar to the ground truth in this task.

7 Conclusion

The project we created was centered around creating a conversational chatbot that allows for interruptions. This better simulates real-life conversation, and allows for a more natural and "human" approach. We drew inspiration from a research paper written in a collaborative effort between UT Austin, Microsoft Research, and AWS to help mold our solution.

We designed and implemented a three-layered approach with a verification model, speculative model, and a response model. The verification model decides when to interrupt, the speculative model completes the incomplete, interrupted user query, and the response model provides the response. With these three components, we created a chatbot that can provide an interrupting feature.

Our evaluation was centered around evaluating both the verification and speculative models. The verification model yielded high accuracies, indicating that it was an adequate model in predicting and recognizing interruptible sentences. The speculative model did not necessarily perform to expectations, but after further examination this did make sense. Language is incredibly complex and large, and accurately predicting a sentence to match the ground truth is an extremely difficult task.

There are several ways to improve our project. For one, we could use larger, more complex models. This would allow us to more accurately predict when to interrupt, and generate better user responses for the speculative model. We could also use more diverse datasets instead of the WikiText and SQuAD datasets. More diverse datasets could allow us to capture different semantic and syntactic patterns, which could allow for a more robust model. We could also add multiple agents in our models to make our chatbot more robust. Drawing inspiration from the paper, "Single-agent or Multi-agent Systems? Why Not Both?", we could utilize multiple agents. The use of multi-agents, while complex, could prove useful in more challenging situations. Multi-agents allow for a broader range of viewpoints as well. One agent could be trained on another set of data than a second agent, allowing for more diverse viewpoints. This is similar to the idea of having multiple experts in a model. We could potentially also leverage this into our project. Essentially, we would need to have a component that determines if a SAS (single agent solution) approach or if a MAS (multi-agent solution) approach would be more beneficial to the current query. We could also extend this, and if the model is unsure of whether to interrupt or not (confidence value close to threshold), then employ another agent to help break this "tie".

We could also further enhance our evaluation process to measure things like latency saved. This would involve calculating the time saved by interrupting and producing an output compared with allowing the user to entirely type in their query. This would prove beneficial as it provides us with a key metric showing us the quantitative benefit of having an interruptible chatbot. Overall, this chatbot is a good start to the solution, but there are improvements.

References

[1] Haoyu Li, Srikanth Kandula, Maria Angels de Luis Balaguer, Aditya Akella, and Venkat Arun. Speculative Ad-hoc Querying.

[2] Beidi Chen Song Han Mike Lewis Guangxuan Xiao, Yuandong Tian. Efficient Streaming Language Models with Attention Sinks.

[3] JinYeong Bak JiWoo Kim, Minsuk Chang. Beyond Turn-taking: Introducing Text-based Overlap into Human-LLM Interactions.

[4] Sandy J. J. Gould Benjamin R. Cowan Justin Edwards, Christian P. Janssen. Eliciting Spoken Interruptions to Inform Proactive Speech Agent Design.

[5] David Schlangen Casey Kennington, Pierre Lison. Incremental Dialogue Management: Survey, Discussion, and Implications for HRI.

[6] Bahar Irfan Gabriel Skantze. Applying General Turn-Taking Models to Conversational Human-Robot Interaction.

[7] Mingyan Gao, Yanzi Li, Banruo Liu, Yifan Yu, Phillip Wang, Ching-Yu Lin, and Fan Lai. Single-agent or Multi-agent Systems? Why Not Both?